

# IEEC YA CONNECT

## ARCHITECTURE HANDBOOK

*Platform Design Authority and Engineering Standards*

|                         |                                 |
|-------------------------|---------------------------------|
| <b>Document Owner</b>   | IEEC YA IT Team                 |
| <b>Version</b>          | 0.3                             |
| <b>Status</b>           | Draft - Architecture Foundation |
| <b>Classification</b>   | Internal Use                    |
| <b>Technology Stack</b> | React + Firebase                |
| <b>Last Updated</b>     | July 13, 2026                   |

**Prepared by IEEC YA IT Team**

Architecture and AI Design Support: OpenAI ChatGPT

## Revision History

| Version | Date          | Status | Changes                                                                                                                        | Owner           |
|---------|---------------|--------|--------------------------------------------------------------------------------------------------------------------------------|-----------------|
| 0.1     | July 12, 2026 | Draft  | Chapter 1 added                                                                                                                | IEEC YA IT Team |
| 0.2     | July 12, 2026 | Draft  | Chapter 2 added;<br>handbook<br>consolidated                                                                                   | IEEC YA IT Team |
| 0.3     | July 13, 2026 | Draft  | Chapter 3 added;<br>multi-organization<br>and federated<br>parent model<br>approved; Person<br>and calendar<br>scope clarified | IEEC YA IT Team |

## Table of Contents

1. Chapter 1 - Purpose, Scope, and Design Authority
2. Chapter 2 - Core Design Principles and Shared Platform Engines
3. Chapter 3 - Organization and Tenant Model
4. Chapter 4 - People and Account Model (planned)
5. Chapter 5 - Identity and Authentication (planned)
6. Chapter 6 - Authorization and Permission Engine (planned)
7. Chapter 7 - Dynamic Forms Engine (planned)
8. Chapter 8 - Workflow Engine (planned)
9. Chapter 9 - Ministry Calendar Engine (planned)
10. Chapter 10 - Chat and Collaboration (planned)
11. Chapter 11 - Notifications and Tasks (planned)
12. Chapter 12 - Audit and History (planned)
13. Chapter 13 - Data and Engineering Standards (planned)

# Chapter 1 - Purpose, Scope, and Design Authority

## 1.1 Purpose

This handbook defines the mandatory platform-wide architecture rules for IEEC YA Connect. Every module, feature, developer, and AI coding assistant must follow these rules unless an approved Architecture Decision Record changes them.

## 1.2 Platform Vision

IEEC YA Connect is a centralized ministry management platform for managing people and ministry operations. Its primary mission is to help leaders shepherd people through the ministry journey while maintaining clear accountability, secure access, and reliable history.

Newcomer -> Member -> Minister -> Leadership

## 1.3 Scope

- People and account management
- Ministry teams, positions, and oversight
- Follow-up and attendance
- Calendar, communication, and collaboration
- Permissions and approvals
- Dynamic forms and workflows
- Notifications, tasks, reports, and audit history
- Support for multiple independent organizations and optional parent organizations

## 1.4 Design Authority

The handbook is the platform design authority. If a module specification conflicts with this handbook, the handbook takes precedence unless an approved Architecture Decision Record explicitly changes the standard.

## 1.5 Mandatory Foundation Rules

- One permanent Person record per individual within each organization.
- Each organization owns and isolates its people, users, ministries, teams, groups, calendar, chat, files, reports, roles, and permissions.
- Roles are permission templates, not final authority.
- Default deny and least privilege.
- Technical authority is separate from ministry authority.
- Oversight is separate from team membership.
- One shared operational calendar per organization.
- Parent-level events may affect selected organizations only through explicit scope and policy.
- Chat membership is separate from team membership.
- Forms and workflows are configurable where practical.
- Soft delete by default.
- Important actions and changes are audited.
- React UI restrictions never replace backend enforcement.

## Chapter 2 - Core Design Principles and Shared Platform Engines

### 2.1 Purpose

This chapter defines the reusable engines and engineering principles that every ministry module must use. Modules must extend these shared services rather than building isolated copies.

### 2.2 People-Centered Architecture

Within an organization, the Person record is the center of the ministry data model. Newcomer journeys, membership status, team assignments, attendance, reports, chats, calendar participation, and permissions reference the same organization-owned Person identity. A status change must never create a duplicate Person within the same organization.

### 2.3 Configuration Before Hard-Coding

Ministry processes change over time. Forms, field options, approval steps, deadlines, reminder rules, sensitivity categories, status labels, and role defaults should be configurable whenever that can be done safely and clearly. Configuration must not weaken security or allow invalid workflows.

### 2.4 Permission Engine

The Permission Engine determines who may view or perform each action. Roles provide default permissions, while individual overrides can explicitly grant or deny permissions. Explicit deny takes precedence over grants. Permission changes must be scope-aware, time-aware when configured, and fully audited.

- Default deny
- Role-based default grants
- Individual grant and deny overrides
- Organization, ministry, team, group, workflow, and record scopes
- Oversight-scoped permissions
- Time-based assignments
- Backend enforcement
- Permission source traceability

### 2.5 Workflow Engine

The Workflow Engine manages approvals, state transitions, required steps, and exceptions. It must support configurable one-step or multi-step approval, sequential or parallel steps, return-for-correction, rejection, cancellation, and complete transition history.

### 2.6 Dynamic Form Engine

The Dynamic Form Engine supports configurable forms for newcomer reports, welcome schedules, member profiles, applications, event registration, and future ministry processes. Published forms are versioned so previous submissions always retain the exact structure used at submission time.

- Short text and long text
- Yes/no
- Single-select and multi-select
- Date and time
- Number
- Required or optional fields
- Conditional fields
- Configurable labels and order

- Versioned publication
- Permission-controlled form management

## 2.7 Ministry Calendar Engine

Each organization uses one shared operational calendar. Ministries, teams, and groups create events inside that calendar according to permission and conflict rules. Parent organizations may create parent-level events that are published to selected organizations and may block local scheduling only when the event scope and conflict policy explicitly require it.

## 2.8 Chat and Collaboration Engine

Teams may have multiple chat channels. Chat membership is independent of team membership, allowing selected ministers from other teams to join a channel without receiving team permissions or access to protected ministry records. Chat data remains organization-owned and isolated.

## 2.9 Notification and Task Engine

Notifications and tasks are shared platform services. They support in-app notifications, push notifications, email, and future SMS or WhatsApp integrations. The system must distinguish informational notices from required actions and must track delivery, read status, due dates, and escalation.

## 2.10 Audit Engine

The Audit Engine records important actions, including permission changes, assignments, report submissions, attendance corrections, status changes, workflow decisions, chat membership changes, calendar overrides, and parent-level oversight access. Audit records should be append-only and protected from normal client modification.

## 2.11 Shared Supporting Services

- File and photo storage
- Search and duplicate detection
- Organization configuration
- Profile completion
- Approval templates
- Sensitive-data classification
- Soft deletion and restoration
- Reporting and analytics

## 2.12 Enforcement Layers

- React UI: shows only permitted screens and actions.
- Application service layer: validates workflows and business rules.
- Firestore Security Rules: prevents unauthorized direct reads and writes.
- Cloud Functions or trusted backend: performs privileged operations and audit logging.
- Automated tests: verify both allowed and denied behavior.

## 2.13 Architecture Decision Records

Major architecture choices must be recorded as ADRs. An ADR includes the decision, context, alternatives considered, consequences, status, and approval date. ADRs explain why the platform was designed a certain way and help future developers avoid reversing critical decisions accidentally.

## 2.14 Initial ADR Catalog

| ADR | Decision | Status |
|-----|----------|--------|
|-----|----------|--------|

| ADR-001 | One organization-owned Person record per individual | Approved |
|---------|-----------------------------------------------------|----------|
| ADR-002 | Roles are permission templates                      | Approved |
| ADR-003 | One shared operational calendar per organization    | Approved |
| ADR-004 | Chat membership independent of team membership      | Approved |
| ADR-005 | Dynamic forms as a shared platform service          | Approved |
| ADR-006 | Soft delete and audit by default                    | Approved |
| ADR-007 | Federated multi-organization tenant model           | Approved |

## 2.15 Completion Criteria

A future module is architecture-compliant only when it reuses the shared engines, defines its permissions and data scope, protects sensitive data, preserves history, supports audit, and avoids duplicating person, calendar, chat, workflow, or form infrastructure.

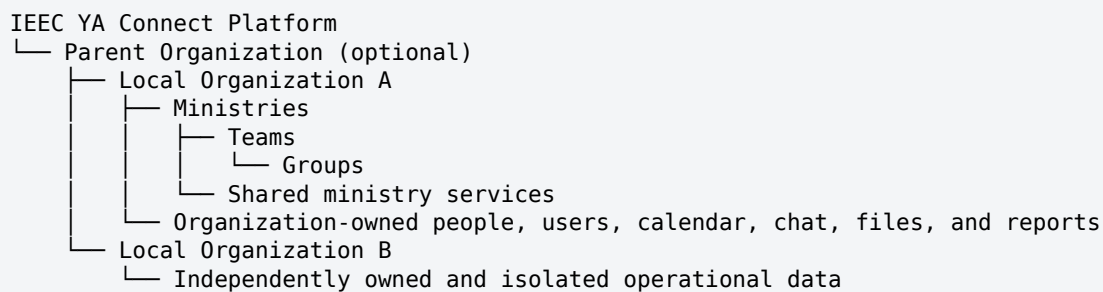
## Chapter 3 - Organization and Tenant Model

### 3.1 Purpose

This chapter defines how IEEC YA Connect hosts multiple organizations, how organizations may be grouped under a parent organization, who owns operational data, and how access is isolated. These rules apply to every collection, permission check, query, file path, background job, report, calendar event, and chat channel.

### 3.2 Approved Tenant Model

IEEC YA Connect will use a federated multi-organization model. Each local organization is an independent tenant that owns and controls its operational data. Multiple local organizations may be connected to an optional parent organization for denomination-wide coordination, standards, aggregate reporting, and explicitly granted oversight.



### 3.3 Core Terms

**Platform:** The IEEC YA Connect software and shared technical infrastructure. Platform administration concerns software operation, not automatic ministry-content access.

**Parent Organization:** An optional denomination, network, or governing body that groups multiple local organizations. It receives only explicitly configured oversight and aggregate access.

**Organization:** An independent church, branch, congregation, or institutional tenant. It owns its people, users, ministries, teams, groups, calendar, chat, files, reports, roles, permissions, and configuration.

**Ministry:** A major organizational area inside one organization, such as Young Adult, Women, Children, Youth, or Worship Ministry.

**Team:** An operational unit inside a ministry, such as Follow-Up, Bible Study, Media, Sound, Worship, or Usher.

**Group:** A subgroup inside a team or ministry, such as a G5 group, Bible Study group, class, cohort, or temporary working group.

### 3.4 Organization Independence

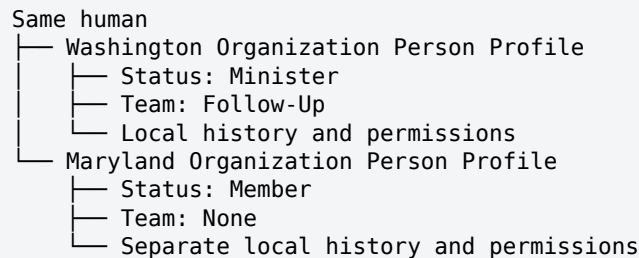
Each organization operates independently even when it belongs to a parent organization. It determines its own ministry structure, people records, status definitions, role templates, permission assignments, forms, workflows, chat channels, calendar events, reports, and local configuration within platform safeguards.

- Organization A cannot read or modify Organization B data by default.
- A local administrator has authority only within assigned organization scope.
- Parent affiliation does not remove local ownership.
- Platform technical roles do not automatically receive pastoral or ministry-content access.

- No local role or permission crosses into another organization unless a separate assignment explicitly grants it.

### 3.5 Organization-Owned People

Each organization owns its own Person records. The same human may exist in more than one organization, but each organization maintains a separate profile, ministry status, history, roles, permissions, attendance, notes, and assignments. Records are not automatically merged or shared across organizations.



A verified sign-in identity may later be associated with access to more than one organization, but this does not combine the organizations' Person records or ministry data. The authentication and organization-access model will be finalized in Chapter 5.

### 3.6 Data Ownership and Required Scope

Every organization-owned operational record must carry an organization scope. Records belonging to a ministry, team, group, workflow, or specific case must also include the relevant lower-level scope identifiers.

- organizationId - required for organization-owned operational data
- parentOrganizationId - used only when relevant to a parent relationship or parent-owned record
- ministryId - required when the record belongs to a ministry
- teamId - required when the record belongs to a team
- groupId - required when the record belongs to a group
- scopeType and scopeId - used for generic permission, configuration, workflow, and reporting services

A document must never rely only on a client-selected path or UI context to determine its organization. Security rules and trusted backend operations must verify the scope from authoritative records.

### 3.7 Parent Organization Oversight

A parent organization may coordinate multiple local organizations, but it does not automatically own or receive unrestricted access to local operational data. Parent access is permission-based, purpose-limited, and auditable.

#### 3.7.1 Typical Parent-Level Capabilities

- View the list and status of affiliated organizations
- View approved aggregate statistics
- Publish denomination-wide announcements
- Create parent-level programs and calendar events
- Maintain shared policies, templates, or standards
- Receive selected reports submitted by local organizations
- Assign specifically scoped oversight responsibilities

#### 3.7.2 Restricted by Default

- Pastoral and sensitive bio notes
- Individual newcomer follow-up reports

- Private local chat channels
- Local files and attachments
- Local permission details
- Individual financial records
- Any person-level data not required for an approved parent function

### 3.8 Scope Types

The permission and configuration engines must understand these scopes:

| Scope               | Purpose                                      | Example                                   |
|---------------------|----------------------------------------------|-------------------------------------------|
| platform            | Technical operation of the software platform | Create or suspend organization tenants    |
| parent_organization | Denomination or network coordination         | View approved aggregate reports           |
| organization        | One local church or tenant                   | Manage local ministries and people        |
| ministry            | One ministry inside an organization          | Manage Young Adult ministry configuration |
| team                | One operational team                         | Assign Follow-Up newcomers                |
| group               | One subgroup                                 | Manage one Bible Study group              |
| workflow            | One workflow instance or workflow type       | Approve a membership recommendation       |
| record              | One specific protected record                | View one restricted escalation            |

### 3.9 Organization Structure

The default IEEC structure is:

```
Parent Organization -> Organization -> Ministry -> Team -> Group
```

The platform may support configurable labels and optional levels in the future, but the initial implementation should preserve these explicit concepts because they carry different business and security meanings. A Group is not the same as a Team, and a Parent Organization is not the same as a local Organization.

### 3.10 Local Calendar and Parent Events

Each organization has one shared operational calendar. All local ministries, teams, and groups create events inside that calendar according to permission and conflict rules. A parent organization may maintain parent-owned events and publish them to selected organizations.

- Parent events must declare affected organizations.
- Parent events must declare whether they are informational, warning-only, approval-required, or hard-blocking.

- A parent event must not silently delete an existing local event.
- When a new parent event creates a conflict, affected local event owners must be notified and the conflict must enter a review process.
- Parent calendar overrides and exceptions must be audited.

### 3.11 Chat and Collaboration Boundaries

Local organization chats remain local and private by default. Parent organizations may create parent-level channels for selected participants from affiliated organizations. Joining a parent channel does not grant access to the participant's local organization data, and joining a local chat does not grant team membership or module permissions.

### 3.12 Roles and Permissions Placement

Chapter 3 defines where authority applies. Chapter 6 will define how roles, permissions, individual overrides, oversight assignments, and explicit denials are resolved. Every assignment must include a scope and must never be interpreted beyond that scope.

```
Person: Daniel
Role template: Follow-Up Leader
Scope: Washington Organization / Young Adult Ministry / Follow-Up Team
Result: Follow-Up management defaults only inside that team, subject to overrides
```

### 3.13 Organization Lifecycle

Organizations must support a controlled lifecycle without deleting operational history.

- draft - tenant setup is incomplete
- active - normal access and operation
- suspended - user access or selected services are temporarily blocked
- archived - organization is no longer active but retained for history and legal requirements
- deleted - exceptional permanent deletion process, restricted and separately governed

Suspension by a platform administrator must not silently transfer ownership or expose local ministry content. The system should allow controlled export, restoration, and retention processes.

### 3.14 Cross-Organization Operations

Cross-organization functions must be explicit shared workflows rather than direct access. Examples include a parent announcement, a denomination-wide event, an aggregate report submission, a shared training registration, or an approved transfer process. Each workflow must define what data is sent, who may receive it, and whether local approval is required.

### 3.15 Security Requirements

- Every request must resolve the active organization context from trusted account membership or assignment data.
- Firestore queries and document reads must verify organization scope.
- Cloud Storage paths and access checks must include organization ownership.
- Background jobs must process records only within declared scope.
- Parent aggregate reports should prefer precomputed or intentionally submitted data instead of unrestricted access to local raw records.
- All parent oversight access and cross-organization data movement must be audited.
- Organization IDs supplied by the client must never be trusted without authorization checks.

### 3.16 ADR-007 - Federated Multi-Organization Tenant Model

|                             |                                                                                                                                                             |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Status</b>               | Approved                                                                                                                                                    |
| <b>Decision</b>             | IEEC YA Connect supports multiple independent organizations grouped under an optional parent organization.                                                  |
| <b>Local ownership</b>      | Each organization owns and isolates its people, users, roles, permissions, ministry records, calendar, chat, files, configuration, and operational history. |
| <b>Parent access</b>        | Parent organizations receive only explicitly configured oversight, aggregate reporting, shared standards, announcements, and event capabilities.            |
| <b>People</b>               | The same human may have separate organization-owned Person profiles. Profiles are not automatically shared or merged.                                       |
| <b>Security consequence</b> | Every operational record and permission assignment must be organization-scoped, and cross-organization access is denied by default.                         |
| <b>Reason</b>               | This supports denomination-wide coordination without sacrificing local ownership, privacy, or independent ministry operations.                              |

### 3.17 Chapter Completion Criteria

- The platform recognizes Platform, Parent Organization, Organization, Ministry, Team, and Group as distinct scopes.
- Every local operational record belongs to one organization.
- Organization-owned people and users are isolated from other organizations.
- Parent access is explicit, limited, and audited.
- Each organization uses one shared operational calendar.
- Parent events can affect selected organizations only through declared policies.
- Roles and permissions never cross scope automatically.
- Cross-organization workflows transfer only approved data.

### Next Chapter

Chapter 4 will define the People and Account Model, including organization-owned Person profiles, newcomer-to-leadership progression, user accounts, invitations, returning people, duplicate detection, profile completion, status history, and access to multiple organizations.